

Transformer - Technical Details

Gaspard Krief

1 Introduction

This project is a complete implementation of the transformer architecture from scratch in C++, with no reliance on machine learning frameworks. All operations (forward passes, backward passes, gradient computation and parameter updates) are implemented manually. Matrix operations are accelerated using Apple's MLX framework for GPU computation on Apple Silicon.

The implementation was tested on English-to-French translation, trained on sentence pairs from the OPUS corpus [3]. The model follows the encoder-decoder architecture introduced in *Attention is All You Need* [4].

The primary motivation for this project was to develop a deeper understanding of the transformer architecture and the attention mechanism by implementing every component from first principles. Building the forward and backward passes manually, rather than relying on automatic differentiation, requires deriving and implementing the gradients for each layer individually. A secondary goal was to explore the constraints of working without a framework: managing memory layouts, implementing numerically stable operations and optimizing for GPU execution through batching and dynamic sequence trimming.

Large language models were used as a tool throughout the project: for C++ best practices, debugging assistance, improving the writing of this document and data preprocessing (extracting and formatting the training corpus). The mathematical derivations and the large majority of the code were done independently.

2 Architecture Overview

The transformer architecture used in this project follows the encoder-decoder design introduced in *Attention is All You Need* [4], with one modification: layer normalization is applied *before* each sublayer (Pre-LN) rather than after, which improves training stability.

The network is composed of the following components:

- **Input Layer:** maps token IDs to their embeddings and adds sinusoidal positional encodings.
- **Layer Normalization:** normalizes each token's embedding.
- **Multi-Head Attention:** computes attention in three configurations:
 - Encoder self-attention: each token attends to all other encoder tokens.
 - Decoder masked self-attention: each decoder token attends only to previous positions.
 - Decoder cross-attention: decoder tokens attend to the full encoder output.
- **Feed-Forward Layer:** applies two linear transformations with a non-linear activation in between.
- **Linear Layer:** projects the final decoder output to logits over the vocabulary.

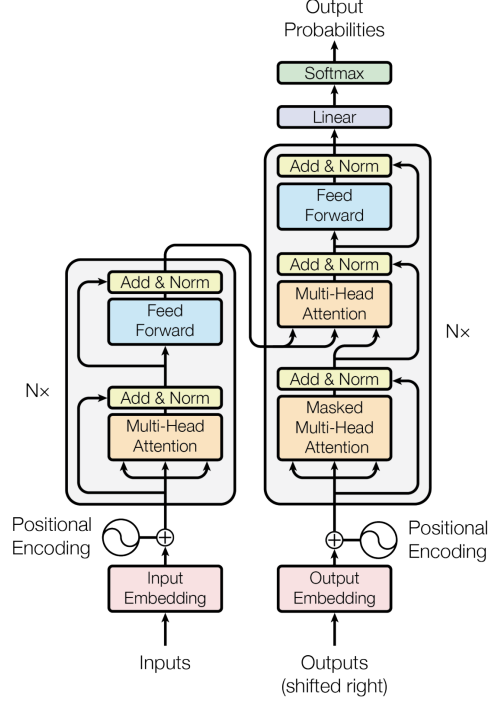


Figure 1: The transformer architecture [4]. This implementation uses Pre-LN, i.e., layer normalization is applied before each sublayer rather than after.

The encoder consists of N identical blocks, each containing a self-attention sublayer followed by a feed-forward sublayer, both wrapped in Pre-LN residual connections. The decoder consists of N identical blocks with an additional cross-attention sublayer between the two.

3 Input Processing

3.1 Tokenization

Raw sentences are tokenized using Byte Pair Encoding (BPE) [2], a subword tokenization algorithm that iteratively merges the most frequent character pairs in the corpus until a target vocabulary size is reached. The tokenizer is trained jointly on the English and French sentences with a shared vocabulary of V tokens, which includes three special tokens: **< sos >** (start of sentence), **< eos >** (end of sentence) and **< pad >** (padding).

3.2 Embeddings and Positional Encodings

Each token ID is mapped to a vector of dimension d_{model} via a learned embedding matrix $E \in \mathbb{R}^{V \times d_{\text{model}}}$. Positional information is injected by adding fixed sinusoidal positional encodings to each embedding:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (1)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

where pos is the position in the sequence and i is the dimension index. The encoder and decoder each have independent embedding matrices but share the same positional encoding scheme.

4 Layers

Throughout this section, we use the shorthand $dx := \frac{\partial \mathcal{L}}{\partial x}$ to denote the gradient of the loss with respect to a quantity x .

4.1 Input Layer

The input layer maps a sequence of token IDs to their embedding vectors and adds positional information. Input sequences are padded to a fixed maximum length $T = \text{seq}$, though in practice sequences within a batch are trimmed to the length of the longest sequence in that batch.

4.1.1 Forward Pass

Let $t \in \mathbb{N}^T$ be a sequence of token IDs. Each token is mapped to its embedding vector by looking up row t_i of E :

$$X_i = E_{t_i,:} \quad i = 1, \dots, T \quad (3)$$

The sinusoidal positional encodings $P \in \mathbb{R}^{T \times d_{\text{model}}}$ defined in Section 3 are then added:

$$\text{out} = X + P \quad (4)$$

The positional encodings are not learned. The embedding of the `<pad>` token is fixed to zero and is not updated during training.

4.1.2 Backward Pass

Let $d\text{out} \in \mathbb{R}^{T \times d_{\text{model}}}$ be the gradient received from the upstream layer. Since P is fixed, the gradient flows only through X . The gradient with respect to E is sparse: only the rows corresponding to tokens present in the input receive a non-zero gradient. Specifically:

$$dE_{v,:} = \sum_{i: t_i=v} d\text{out}_{i,:} \quad v = 0, \dots, V-1 \quad (5)$$

That is, for each vocabulary entry v , its embedding gradient is the sum of $d\text{out}$ rows at all positions where token v appears in the input. The gradient of the `<pad>` embedding is set to zero regardless.

4.2 Layer Normalization

Layer normalization normalizes each token’s embedding independently across the feature dimension.

4.2.1 Forward Pass

Given the input $X \in \mathbb{R}^{T \times d_{\text{model}}}$, we compute:

$$\mu_i = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} X_{i,j} \quad (6)$$

$$\sigma_i^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (X_{i,j} - \mu_i)^2 \quad (7)$$

$$\hat{X} = \frac{X - \mu}{\sqrt{\sigma^2 + \varepsilon}} \quad (8)$$

$$\bar{X} = \hat{X} \odot \gamma + \beta \quad (9)$$

where $\mu \in \mathbb{R}^T$ and $\sigma^2 \in \mathbb{R}^T$ are the per-token mean and variance, with μ and σ^2 broadcast along the feature dimension in the normalization step, $\gamma, \beta \in \mathbb{R}^{d_{\text{model}}}$ are learned scale and shift parameters shared across all tokens and broadcast along the sequence dimension and ε is a small constant for numerical stability.

4.2.2 Backward Pass

Let $d\bar{X} \in \mathbb{R}^{T \times d_{\text{model}}}$ be the gradient received from the upstream layer during backpropagation.

$$d\gamma = \sum_{i=1}^T d\bar{X}_{i,:} \odot \hat{X}_{i,:} \quad (10)$$

$$d\beta = \sum_{i=1}^T d\bar{X}_{i,:} \quad (11)$$

$$d\hat{X} = d\bar{X} \odot \gamma \quad (12)$$

The gradient dX is less straightforward to derive since $X_{i,j}$ influences all outputs $\hat{X}_{i,:}$ through the shared mean and variance. The full derivation is given in Appendix A. Defining $A = d\hat{X} \mathbf{1}_{d_{\text{model}}}^\top \in \mathbb{R}^T$ and $B = (d\hat{X} \odot \hat{X}) \mathbf{1}_{d_{\text{model}}}^\top \in \mathbb{R}^T$, the result is:

$$dX = s \odot \left(d\hat{X} - \frac{1}{d_{\text{model}}} \left(\hat{X} \odot B \mathbf{1}_{d_{\text{model}}}^\top + A \mathbf{1}_{d_{\text{model}}}^\top \right) \right) \quad (13)$$

where $s \in \mathbb{R}^T$ with $s_i = (\sigma_i^2 + \varepsilon)^{-1/2}$, broadcast along the feature dimension.

4.3 Multi-Head Attention

Multi-head attention computes attention in parallel across h independent subspaces, allowing the model to capture different types of relationships between tokens simultaneously.

4.3.1 Forward Pass

Let $X_Q \in \mathbb{R}^{T_Q \times d_{\text{model}}}$ and $X_{KV} \in \mathbb{R}^{T_{KV} \times d_{\text{model}}}$ be the query and key-value inputs respectively, where T_Q and T_{KV} denote the query and key-value sequence lengths. For self-attention, $X_{KV} = X_Q$. Given weight matrices $W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times hd_k}$, $W^V \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, the forward pass is:

$$Q = X_Q W^Q \in \mathbb{R}^{T_Q \times hd_k} \quad (14)$$

$$K = X_{KV} W^K \in \mathbb{R}^{T_{KV} \times hd_k} \quad (15)$$

$$V = X_{KV} W^V \in \mathbb{R}^{T_{KV} \times hd_v} \quad (16)$$

Q , K and V are then reshaped to $\mathbb{R}^{h \times T_Q \times d_k}$, $\mathbb{R}^{h \times T_{KV} \times d_k}$ and $\mathbb{R}^{h \times T_{KV} \times d_v}$ respectively to separate the heads. The attention scores for each head are first computed as:

$$\text{scores}_l = \frac{Q_l K_l^\top}{\sqrt{d_k}} + M \in \mathbb{R}^{T_Q \times T_{KV}} \quad (17)$$

so that $S_l := \text{softmax}(\text{scores}_l)$ and $\text{head}_l = S_l V_l \in \mathbb{R}^{T_Q \times d_v}$. The softmax is applied row-wise i.e. independently for each query position.

$M \in \mathbb{R}^{T_Q \times T_{KV}}$ is an optional mask matrix. It is set to $-\infty$ at positions that should not be attended to and 0 elsewhere. Three masking configurations are used:

- Encoder self-attention: $M = 0$ everywhere except at padding positions.
- Decoder masked self-attention: M combines a causal mask (upper triangle set to $-\infty$) with padding positions.
- Decoder cross-attention: $M = 0$ everywhere except at encoder padding positions.

The outputs of all heads are concatenated and projected:

$$\text{out} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \in \mathbb{R}^{T_Q \times d_{\text{model}}} \quad (18)$$

4.3.2 Backward Pass

Let $d\text{out} \in \mathbb{R}^{T_Q \times d_{\text{model}}}$ be the gradient received from the upstream layer during backpropagation. Let $C = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{T_Q \times hd_v}$ denote the concatenated heads. Working backwards through the output projection:

$$dC = d\text{out} (W^O)^\top \quad (19)$$

$$dW^O = C^\top d\text{out} \quad (20)$$

After reshaping dC back to $\mathbb{R}^{h \times T_Q \times d_v}$, the following applies independently for each head $l = 1, \dots, h$.

$$dS_l = d\text{head}_l V_l^\top \quad (21)$$

$$dV_l = S_l^\top d\text{head}_l \quad (22)$$

The softmax backward applied row-wise to dS_l gives:

$$dscores_l = S_l \odot \left(dS_l - (dS_l \odot S_l) \mathbf{1}_{T_{KV}} \mathbf{1}_{T_{KV}}^\top \right) \quad (23)$$

where $(dS_l \odot S_l) \mathbf{1}_{T_{KV}}$ computes the row-wise dot product, broadcast back across the T_{KV} dimension. For masked self-attention, entries of $dscores_l$ corresponding to masked positions are set to zero, since those positions were blocked in the forward pass and have no gradient path. We then get:

$$dQ_l = dscores_l \frac{K_l}{\sqrt{d_k}} \quad (24)$$

$$dK_l = dscores_l^\top \frac{Q_l}{\sqrt{d_k}} \quad (25)$$

Finally, reshaping $dQ^{\text{flat}} \in \mathbb{R}^{T_Q \times hd_k}$, $dK^{\text{flat}} \in \mathbb{R}^{T_{KV} \times hd_k}$, $dV^{\text{flat}} \in \mathbb{R}^{T_{KV} \times hd_v}$ as the per-head gradients concatenated along the feature dimension, we project back through the weight matrices:

$$dW^Q = X_Q^\top dQ^{\text{flat}} \quad (26)$$

$$dW^K = X_{KV}^\top dK^{\text{flat}} \quad (27)$$

$$dW^V = X_{KV}^\top dV^{\text{flat}} \quad (28)$$

$$dX_Q = dQ^{\text{flat}} (W^Q)^\top \quad (29)$$

$$dX_{KV} = dK^{\text{flat}} (W^K)^\top + dV^{\text{flat}} (W^V)^\top \quad (30)$$

For self-attention $X_Q = X_{KV}$, so $dX = dX_Q + dX_{KV}$. For cross-attention, dX_Q flows back to the decoder and dX_{KV} flows back to the encoder.

4.4 Feed-Forward Layer

The feed-forward layer applies two linear transformations with a non-linear activation in between, independently to each token.

4.4.1 Forward Pass

Given input $X \in \mathbb{R}^{T \times d_{\text{model}}}$:

$$U = XW_1 + b_1 \quad (31)$$

$$H = \sigma(U) \quad (32)$$

$$\text{out} = HW_2 + b_2 \quad (33)$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$, $b_1 \in \mathbb{R}^{d_{ff}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$, $d_{ff} = 4 \cdot d_{\text{model}}$ and σ is a differentiable activation function. In this implementation, ReLU is used: $\sigma(x) = \max(0, x)$.

4.4.2 Backward Pass

Let $dout \in \mathbb{R}^{T \times d_{\text{model}}}$ be the gradient received from the upstream layer during backpropagation.

$$dW_2 = H^\top dout \quad (34)$$

$$db_2 = \sum_{i=1}^T dout_{i,:} \quad (35)$$

$$dH = dout W_2^\top \quad (36)$$

$$dU = dH \odot \sigma'(U) \quad (37)$$

$$dW_1 = X^\top dU \quad (38)$$

$$db_1 = \sum_{i=1}^T dU_{i,:} \quad (39)$$

$$dX = dU W_1^\top \quad (40)$$

4.5 Linear Layer

The final linear layer projects the decoder output to logits over the vocabulary.

4.5.1 Forward Pass

Given input $X \in \mathbb{R}^{T \times d_{\text{model}}}$:

$$\text{out} = XW + b \quad (41)$$

where $W \in \mathbb{R}^{d_{\text{model}} \times V}$, $b \in \mathbb{R}^V$, and V is the vocabulary size.

4.5.2 Backward Pass

Let $dout \in \mathbb{R}^{T \times V}$ be the gradient of the loss with respect to the logits.

$$dW = X^\top dout \quad (42)$$

$$db = \sum_{i=1}^T dout_{i,:} \quad (43)$$

$$dX = dout W^\top \quad (44)$$

5 Training

5.1 Cross-Entropy Loss

Given the predicted logits $Z \in \mathbb{R}^{T \times V}$ and the target token IDs $y \in \mathbb{N}^T$, the cross-entropy loss is computed as:

$$\mathcal{L} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log p_{i,y_i} \quad (45)$$

where $\mathcal{M} = \{i : y_i \neq \langle \text{pad} \rangle\}$ is the set of non-padding positions and $p_{i,:} = \text{softmax}(Z_{i,:})$.

Computing $\log p_{i,y_i} = \log \text{softmax}(Z_{i,:})_{y_i}$ directly is numerically unstable due to the exponentials. Instead, we use the log-sum-exp trick (see Appendix B):

$$\log p_{i,y_i} = Z_{i,y_i} - \max_j Z_{i,j} - \log \sum_j \exp(Z_{i,j} - \max_j Z_{i,j}) \quad (46)$$

Subtracting $\max_j Z_{i,j}$ before exponentiating prevents overflow while leaving the result unchanged.

The gradient of the loss with respect to the logits (derived in Appendix C) is:

$$dZ_{i,:} = \begin{cases} \frac{1}{|\mathcal{M}|} (p_{i,:} - e_{y_i}) & \text{if } i \in \mathcal{M} \\ 0 & \text{otherwise} \end{cases} \quad (47)$$

where $e_{y_i} \in \mathbb{R}^V$ is the one-hot vector for the target token at position i .

5.2 Backpropagation

Gradients flow backwards through the network in the following order: the loss gradient dZ is passed to the linear layer, whose output gradient is then passed through the decoder stack from last to first. Each decoder block receives the gradient from the block above and propagates it both to the previous decoder block and to the encoder stack (via the cross-attention path). Since each of the N decoder blocks has a cross-attention sublayer attending to the encoder output, the total gradient flowing back to the encoder is the sum of contributions from all N decoder blocks. This accumulated gradient is then passed through the encoder stack from last to first and finally through the encoder input layer.

5.3 Adam Optimizer

Parameters are updated using the Adam optimizer [1]. At each step t , for each parameter θ with gradient g_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (48)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (49)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (50)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (51)$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \quad (52)$$

where m_t and v_t are the first and second moment estimates, $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected estimates, η is the learning rate, $\beta_1, \beta_2 \in [0, 1)$ are the decay rates and ε is a small constant for numerical stability.

5.4 Batching Strategy

Training sentences are grouped into batches of size B . Since sentences within a batch have varying lengths, they are padded to the length of the longest sentence in the batch rather than to the global maximum sequence length T . This dynamic trimming reduces unnecessary computation on padding tokens, particularly in the attention layers where the cost scales quadratically with sequence length. Batches are shuffled at the start of each epoch to improve gradient diversity.

6 Inference

At inference time, the encoder processes the full source sentence in a single forward pass. The decoder then generates the translation autoregressively: starting from the `<sos>` token, it predicts one token at a time by taking the argmax of the output logits at the last position, appending it to the sequence and feeding the grown sequence back into the decoder. This continues until the `<eos>` token is produced or the maximum sequence length is reached.

7 Results

The goal of this project was not to produce a competitive translation system, but to validate the correctness of the implementation by training a functional translator.

7.1 Model Configuration

Parameter	Value
Encoder layers (N)	4
Decoder layers (N)	4
Model dimension (d_{model})	512
Attention heads (h)	8
Key/value dimension ($d_k = d_v$)	64
Feed-forward dimension (d_{ff})	2048
Vocabulary size (V)	16 000
Maximum sequence length	60
Total parameters	~54M

Table 1: Network configuration.

7.2 Training Setup

The model was trained on approximately 370 000 English–French sentence pairs from the OPUS corpus [3], with a 95/5 train/validation split. No held-out test set was used: model quality was evaluated qualitatively on manually chosen sentences not present in the training data. Training was performed on a 2023 MacBook Pro with an Apple M3 chip, with each epoch taking approximately 1 hour and 15 minutes.

The Adam optimizer was used with $\beta_1 = 0.9$, $\beta_2 = 0.999$ and a batch size of 256. The learning rate was set to 0.0003 for epochs 0-5 and reduced to 0.0001 for the remaining epochs.

7.3 Loss Curve

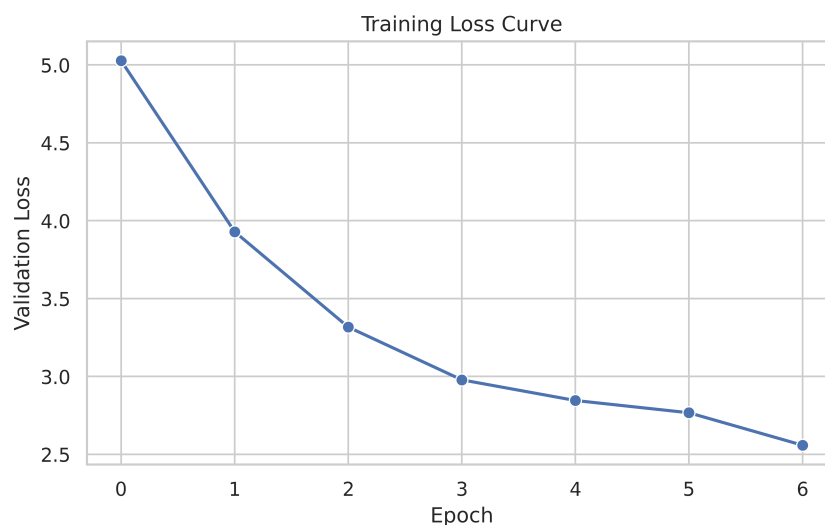


Figure 2: Validation loss over training epochs.

7.4 Example Translations

English (input)	French (model output)	Assessment
The president spoke to the press.	le président a parlé à la presse.	Correct
Oil prices have fallen this week.	les prix pétroliers ont chuté cette semaine.	Correct
The government announced new measures to reduce unemployment.	le gouvernement a annoncé des nouvelles mesures pour réduire le chômage.	Correct
Scientists have discovered a new treatment for the disease.	les chercheurs ont découvert un nouveau traitement pour la maladie.	Correct
She said that she would come tomorrow.	elle a dit qu'elle viendrait demain.	Correct
The children are playing in the garden.	les enfants jouent dans le jardin.	Correct
The train leaves at half past three.	le train se laisse au moins 3 mai.	Incorrect
The doctor told me to rest for a week.	le docteur m'a dit de me reposer.	Partial — missing "for a week"
The United States and France signed a new agreement.	les États-unis et la france ont signé un nouvel accord.	Correct
Paris is the capital of France.	paris est le capital de france.	Minor error — "le capital" should be "la capitale" (gender)
Thank you for your help.	merci pour votre aide.	Correct
I don't understand what you're saying.	je ne comprends pas ce que tu dis.	Correct

Table 2: Example translations produced by the model after 7 epochs of training (final validation loss: 2.56).

7.5 Analysis

Out of 12 simple test sentences, the model produces 9 fully correct translations, 1 with a minor grammatical error, 1 partially correct and 1 incorrect. The model handles a range of grammatical structures well, including the conditional tense (“she said she would come tomorrow” → “elle a dit qu’elle viendrait demain”), negation (“I don’t understand what you’re saying” → “je ne comprends pas ce que tu dis”) and proper nouns (“The United States and France” → “les États-unis et la france”).

The main failure (“The train leaves at half past three”) involves a time expression that is likely underrepresented in the training data. The model also occasionally drops parts of a sentence (“for a week” omitted from the doctor sentence) and makes gender agreement errors (“le capital” instead of “la capitale”).

These results confirm that the implementation is correct and that the model is able to learn meaningful translation capabilities from the training data. The remaining errors are consistent with the constraints of the project: a vocabulary of 16 000 BPE tokens, approximately 370 000 training pairs and 7 training epochs on a single consumer laptop.

A Derivation of the Layer Normalization Backward Pass

We derive the gradient dX of the layer normalization backward pass. Let $s_i = (\sigma_i^2 + \varepsilon)^{-1/2}$, so that $\hat{X}_{i,k} = s_i(X_{i,k} - \mu_i)$, equivalently $X_{i,k} - \mu_i = \hat{X}_{i,k}/s_i$.

Since $X_{i,j}$ influences all normalized outputs $\hat{X}_{i,:}$ through the shared mean μ_i and variance σ_i^2 , the chain rule gives:

$$dX_{i,j} = \sum_k d\hat{X}_{i,k} \cdot \frac{\partial \hat{X}_{i,k}}{\partial X_{i,j}} \quad (53)$$

We then get:

$$\frac{\partial \hat{X}_{i,k}}{\partial X_{i,j}} = \frac{\partial s_i}{\partial X_{i,j}}(X_{i,k} - \mu_i) + s_i \cdot \frac{\partial (X_{i,k} - \mu_i)}{\partial X_{i,j}} \quad (54)$$

First term. Computing $\frac{\partial s_i}{\partial X_{i,j}}$:

$$\frac{\partial s_i}{\partial X_{i,j}} = -\frac{1}{2}(\sigma_i^2 + \varepsilon)^{-3/2} \cdot \frac{\partial \sigma_i^2}{\partial X_{i,j}} \quad (55)$$

$$= -\frac{s_i^3}{2} \cdot \frac{2(X_{i,j} - \mu_i)}{d_{\text{model}}} \quad (56)$$

$$= -\frac{s_i^3(X_{i,j} - \mu_i)}{d_{\text{model}}} \quad (57)$$

Substituting $X_{i,k} - \mu_i = \hat{X}_{i,k}/s_i$ and $X_{i,j} - \mu_i = \hat{X}_{i,j}/s_i$:

$$\frac{\partial s_i}{\partial X_{i,j}}(X_{i,k} - \mu_i) = -\frac{s_i^3(X_{i,j} - \mu_i)}{d_{\text{model}}} \cdot \frac{\hat{X}_{i,k}}{s_i} = -\frac{s_i \hat{X}_{i,j} \hat{X}_{i,k}}{d_{\text{model}}} \quad (58)$$

Second term. Since $\mu_i = \frac{1}{d_{\text{model}}} \sum_j X_{i,j}$:

$$\frac{\partial(X_{i,k} - \mu_i)}{\partial X_{i,j}} = \delta_{ik} - \frac{1}{d_{\text{model}}} \quad (59)$$

Combining. Plugging both terms back:

$$\frac{\partial \hat{X}_{i,k}}{\partial X_{i,j}} = s_i \left(\delta_{ik} - \frac{1}{d_{\text{model}}} - \frac{\hat{X}_{i,j} \hat{X}_{i,k}}{d_{\text{model}}} \right) \quad (60)$$

Substituting into the chain rule sum:

$$dX_{i,j} = s_i \sum_k d\hat{X}_{i,k} \left(\delta_{ik} - \frac{1}{d_{\text{model}}} - \frac{\hat{X}_{i,j} \hat{X}_{i,k}}{d_{\text{model}}} \right) \quad (61)$$

$$= s_i \left(d\hat{X}_{i,j} - \frac{1}{d_{\text{model}}} \sum_k d\hat{X}_{i,k} - \frac{\hat{X}_{i,j}}{d_{\text{model}}} \sum_k d\hat{X}_{i,k} \hat{X}_{i,k} \right) \quad (62)$$

Defining $A_i = \sum_k d\hat{X}_{i,k}$ and $B_i = \sum_k d\hat{X}_{i,k} \hat{X}_{i,k}$:

$$dX_{i,j} = s_i \left(d\hat{X}_{i,j} - \frac{1}{d_{\text{model}}} (\hat{X}_{i,j} \cdot B_i + A_i) \right) \quad (63)$$

which in matrix form gives the expression stated in the main text.

B Derivation of the Log-Sum-Exp Trick

By definition of softmax:

$$\begin{aligned} \log p_{i,y_i} &= \log \frac{\exp(Z_{i,y_i})}{\sum_j \exp(Z_{i,j})} \\ &= Z_{i,y_i} - \log \sum_j \exp(Z_{i,j}) \end{aligned} \quad (64)$$

Let $m_i = \max_j Z_{i,j}$. Since $\exp(Z_{i,j}) = \exp(m_i) \cdot \exp(Z_{i,j} - m_i)$:

$$\begin{aligned} \log \sum_j \exp(Z_{i,j}) &= \log \left(\exp(m_i) \sum_j \exp(Z_{i,j} - m_i) \right) \\ &= m_i + \log \sum_j \exp(Z_{i,j} - m_i) \end{aligned} \quad (65)$$

Substituting back:

$$\log p_{i,y_i} = Z_{i,y_i} - m_i - \log \sum_j \exp(Z_{i,j} - m_i) \quad (66)$$

The terms inside the exponentials are now all ≤ 0 , preventing overflow.

C Derivation of the Cross-Entropy Gradient

Using $\log p_{i,y_i} = Z_{i,y_i} - \log \sum_j \exp(Z_{i,j})$, the derivative with respect to $Z_{i,k}$ is:

$$\frac{\partial \log p_{i,y_i}}{\partial Z_{i,k}} = \delta_{k,y_i} - \frac{\exp(Z_{i,k})}{\sum_j \exp(Z_{i,j})} = \delta_{k,y_i} - p_{i,k} \quad (67)$$

where δ_{k,y_i} is the Kronecker delta. Substituting into the loss:

$$dZ_{i,k} = \frac{\partial \mathcal{L}}{\partial Z_{i,k}} = -\frac{1}{|\mathcal{M}|}(\delta_{k,y_i} - p_{i,k}) = \frac{1}{|\mathcal{M}|}(p_{i,k} - \delta_{k,y_i}) \quad (68)$$

which in vector form gives $dZ_{i,:} = \frac{1}{|\mathcal{M}|}(p_{i,:} - e_{y_i})$ for $i \in \mathcal{M}$ and $dZ_{i,:} = 0$ otherwise.

References

- [1] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [2] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.
- [3] Jörg Tiedemann and Lars Nygaard. The opus corpus-parallel and free: <http://logos.uio.no/opus>. In *LREC*, 2004.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.